EBU6307 – Visual Computing – 2025/26

Coursework report and exercises

Name: _____

QM student number: _____

BUPT student number: _____

DISCLAIMER

- The students are only allowed to use the provided Anaconda environment or GPU jupyter instance.
- All codes will be tested in the provided GPU instance environment.
- All codes should be executable in the terminal. That is, `python <EXERCISE_NAME>.py` (e.g. `python Objective_1.py`) should be able to produce all the results.
- Outputs of the implementation must be generated in the **\results** folder
- Zero mark will be given if your code is not executable in the provided Anaconda environment.
- Zero mark will be given if another student's name and ID are written in your own dataset, and **the submitted coursework will be reported as a plagiarism case**.
- If the submitted file does not follow the structure below, **10 marks will be deducted**.
- Make your folder structure as below:

```
EBU6307_FIRSTNAME_FAMILYNAME_QMSTUDNETNUMBER
├── results
└── codes
```

- In the final coursework submission, submit 1) your report and the single 2) zip file of the above folder (`EBU6307_FIRSTNAME_FAMILYNAME_QMSTUDNETNUMBER`) to the QMplus.
- QMplus submission example:
  - EBU6307_SALMAN_HALEEM_1911XXXXX.pdf
  - EBU6307_SALMAN_HALEEM_1911XXXXX.zip

Project Title: <u>Depth Estimation through stereo images</u>

1. Introduction & Motivation

Depth estimation through stereo images is a computer vision technique that mimics human binocular vision by using two cameras placed at a fixed distance apart to infer the three-dimensional structure of a scene. By capturing simultaneous left and right images and analyzing the slight positional differences of objects between them—known as disparity—a stereo vision system can compute depth using geometric relationships involving the cameras' focal length and baseline distance. This method is widely valued because it is passive, cost-effective, and capable of producing dense depth maps without emitting light or relying on specialized sensors, making it suitable for applications such as autonomous driving, robotics navigation, obstacle detection, 3D reconstruction, and augmented reality. Despite its advantages, depth estimation using stereo images poses challenges such as handling textureless areas, repetitive patterns, lighting variations, and occlusions, which have driven the development of both traditional algorithms like block matching and semi-global matching, as well as advanced deep-learning-based stereo networks that significantly improve correspondence accuracy and robustness.

2. Description

There are four objectives of this mini project. The algorithm developed for each objective will be evaluated via standard metrics.

<u>O1: Generate the synthetic images using the concepts of Image Warping</u> **(20 marks)**

One effective approach for creating synthetic stereo images is image warping, a geometric transformation technique that repositions pixels based on a defined mapping function. By applying controlled horizontal displacements that simulate the viewpoint shift between a left and right camera, a single 2D image can be transformed into a pair of stereo images. This process approximates the parallax effect: objects closer to the camera shift more than distant objects, creating the depth cues necessary for stereo perception. Through these transformations, synthetic stereo images can be produced efficiently and consistently, providing valuable data for depth estimation, 3D reconstruction, and machine learning applications.

<u>Task:</u> Your task is to create an algorithm which can generate synthetic dataset of these stereo images along with the disparity map which can serve as ground truth for training these images

<u>Evaluation Metrics:</u> One of the possible metrics can be Structural Similarity Index Measure (SSIM). This can be expressed as

$$\mathrm{SSIM}(x,y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

Where μ and σ represent mean and standard deviation respectively of image x and y; σ_{xy} denotes covariance; $c_1 = k_1 L$ and $c_2 = k_2 L$ are variables to stabilize the division with weak denominator; L is dynamic range of pixel values ($2^{bits\ per\ pixel} - 1$); k_1=0.01 and k_1=0.03 by default.

Output:

- Generate pipeline for automatically creating the synthetic images
  `results/O1a_synthetic_data/syn_pipeline.jpg`

- The output has to be generated to be saved in results folder and in the same format as original dataset. For example
  `results/O1b_synthetic_data/O1_chess3/~`

- Generate table representing structural similarity for each type of synthetic image generated
  `results/O1c_synthetic_data/SSIM.csv`

O2: Generate the SIFT Feature Descriptors **(20 marks)**

As discussed in lectures The Scale-Invariant Feature Transform (SIFT) is one of the most influential algorithms in computer vision for detecting and describing local features in images. Its strength lies in its robustness: SIFT features remain stable under changes in scale, rotation, illumination, and even moderate viewpoint variations. Because of this, SIFT is widely used in stereo vision, image stitching, object recognition, and 3D reconstruction.

Task: Your task is to generate the SIFT feature descriptors of stereo images and assess the quality of the generated descriptors. Implement your own SIFT method rather than relying on built in opencv packages

Evaluation Metrics: One way to assess SIFT quality is the repeatability measures how consistently SIFT detects the same keypoints under different conditions. Apply warping developed in O1 and assess the repeatability using the following formula

$$\text{Repeatability} = \frac{\text{Number of repeated key points}}{\text{Minimum number of keypoints in either image}}$$

Output:

- Generate pipeline for generating SIFT features of stereo images
  `results/O2a_sift/sift_pipeline.jpg`

- Generate three examples plotting the SIFT keypoints. Plot atleast 10 key points in each example which are repeated across different transformations. Also display type and extent of transformation (has to be different in each example) on the images. Save the images as
  `results/O2b_sift/example_[#number].jpg`

- Generate table representing SIFT features repeatability for each type of images in the dataset
  `results/O2c_sift/Reapitability.csv`

O3: Stereo Depth Estimation using SIFT features **(25 marks)**

Depth estimation aims to recover the 3D structure of a scene from two or more images. While modern stereo algorithms rely on dense pixel-wise matching, SIFT provides a sparse, feature-based approach. Instead of estimating depth for every pixel, SIFT estimates depth only at distinctive keypoints that can be reliably matched across images. This makes SIFT-based depth estimation useful for different applications such as 3D reconstruction

Task: Your task is to utilize the stereo driven SIFT feature descriptors to estimate the depth of image.

Evaluation Metrics: Evaluate the performance using disparity error given as:

$$disparity\ error = \frac{1}{N}\sum_{i}|d_i - d_i^{gt}|$$

Measures pixel-level disparity error.

Output:

- Generate pipeline for generating Stereo Depth Estimation using SIFT features
  `results/O3a_disparity/dep_pipeline.jpg`

- Generate three examples displaying the disparity map. Save the images as
  `results/O3b_disparity/example_[#number].jpg`

- Generate table representing disparity error on estimating depth of each stereo images in the dataset
  `results/O3c_disparity/disparity.csv`

O4: Stereo Depth Estimation using Vision Transformers **(35 marks)**

Task: Train the Vision Transformers (derive the model of your choice) to estimate the stereo depth. This time, instead of utilizing SIFT, generate feature descriptors using ViT models. Evaulate the performance using 5-fold cross validation in terms of disparity error and RMSE. Compare the performance of depth estimation generated in O3.

Output:

- Generate architecture of deep learning pipeline from python functions training the stereo images
  `results/O4a_transformer/architecture.jpg`

- Generate three examples of predicted disparity map as an output for estimated depth. Also mention which test images have been used to create those disparity maps. Save `results/04b_transformer/ example_[#number].jpg`

- Generate table representing cross validation results in terms of disparity error in each fold of test images `results/04c_transformer/crossvalidation.csv`

3. Dataset description

We have extracted the Middlebury dataset (2021 Mobile stereo datasets with ground truth https://vision.middlebury.edu/stereo/data/scenes2021/). Each dataset consists of 2 views taken under several different illuminations and exposures. The folder contains 24 scenes (image pair, disparities, calibration file). The files are organized as follows:

SCENE{1,2,3}/ -- scene imaged from 1-3 viewing directions

im0e{0,1,2,...}.png -- left view under different exposures

im1e{0,1,2,...}.png -- right view under different exposures

calib.txt -- calibration information

im{0,1}.png -- default left and right view (typically ambient/L0/im{0,1}e2.png)

disp{0,1}.pfm -- left and right GT disparities

Calibration file format with explanation:

Here is a sample calib.txt file:

i. cam0=[1758.23 0 953.34; 0 1758.23 552.29; 0 0 1]
   cam1=[1758.23 0 953.34; 0 1758.23 552.29; 0 0 1]
   cam0,1: camera matrices for the rectified views, in the form [f 0 cx; 0 f cy; 0 0 1], where f: focal length in pixels. cx, cy: principal point

ii. doffs=0 (x-difference of principal points, doffs = cx1 - cx0 (here always == 0))
iii. baseline=111.53 (camera baseline in mm)
iv. width=1920
   height=1080
   width, height: image size
v. ndisp=290 (a conservative bound on the number of disparity levels; the stereo algorithm MAY utilize this bound and search from d = 0 .. ndisp-1)
vi. isint=0
vii. vmin=75
   vmax=262
   vmin, vmax: a tight bound on minimum and maximum disparities, used for color visualization, the stereo algorithm MAY NOT utilize this information

To convert from the floating-point disparity value d [pixels] in the .pfm file to depth Z [mm] the following equation can be used: Z = baseline * f / (d + doffs)

4. Use of GenAI

You are encouraged to use AI to assist various aspects of the project, including brainstorming, prototyping, algorithm pipeline design, coding and evaluation. However, you should not rely solely on AI-generated content, as AI tools have limitations. Outputs may lack domain-specific knowledge, contain subtle logic errors, produce inefficient code, or overlook ethical and privacy considerations. Additionally, AI solutions are often generic and may not reflect the critical thinking and adaptability required for real-world problem-solving. To ensure understanding and avoid over-reliance, you should be prepared to answer questions during live demonstration, such as "Why did AI make this recommendation?" or "How did you modify AI's solution?" to demonstrate your reasoning and modifications.

If you implement AI-powered functions (e.g. skill matching, recommendation), you must integrate AI responsibly and critically. AI-generated outputs must not be blindly accepted. Your system must combine structured logic with AI-based interpretation and provide explainable results.

5. Timeline

BUPT week	Scheduled Lab Day	Slot	Time	Deliverables	Assessment Point	Weight
3	Thursday	slot 5	11:30-12:15			
5	Wednesday	slot 9-11	15:40-18:10	O1 and O2	Demonstration and Viva Working Code	Used for collecting the individual factors
9	Thursday	slot 5	11:30-12:15			
11	Wednesday	slot 9-11	15:40-18:10	O3 and O4	Demonstration and Viva Working Code	Used for collecting the individual factors
13	Friday				Final submission on report and the code on QMPlus.	100%

6. Report Structure

<u>Each objective should describe the report in the following format</u>

a. Algorithm pipeline (block diagram)
b. Three examples generated in each objective saved in the results folder
c. Evaluation results (use tables)
d. Discussion the results (8-10 lines)

7. QMPlus Submission

Make your folder structure as below:

```
EBU6307_FIRSTNAME_FAMILYNAME_QMSTUDNETNUMBER
├── results
└── codes
```

In the final coursework submission, submit 1) your report and the single 2) zip file of the above folder (`EBU6307_FIRSTNAME_FAMILYNAME_QMSTUDNETNUMBER`) to the QMplus.

QMplus submission example:

- EBU6307_SALMAN_HALEEM_1911XXXXX.pdf
- EBU6307_SALMAN_HALEEM_1911XXXXX.zip

Individual Factor (for both slot 5 and slot 11)

Component	**Detail**	**Check**
Code Reflection O1 & O2	Check if student knows what designed in pipeline and the code	1/6
Demo O1 & O2	Testing task. The code should be running even if performance is not ideal. Should be zero if code not running	1/6
Next steps O1 & O2	If not achieved optimum performance, next steps need to be taken to improve performance	1/6
Code Reflection O3 & O4	Check if student knows what designed in pipeline and the code	1/6
Demo O3 & O4	Testing task. The code should be running even if performance is not ideal. Should be zero if code not running	1/6
Next steps O3 & O4	If not achieved optimum performance, next steps need to be taken to improve performance	1/6
Total		**1**